



**POLITÉCNICO
ESTELLA**

ASIGNATURA: DESARROLLO DE INTERFACES

**GRADO SUPERIOR EN DESARROLLO DE
APLICACIONES MULTIPLATAFORMA**

DI05 TAREA

Presentado por: Eugen Moga

Fecha: 02/04/2026

ÍNDICE

Parte A – WPF Informe RDLC.....	1
Parte B – Android Informe PDF	4

Parte A – WPF Informe RDLC

Captura del informe generado.

Informe de Personal – 02/04/2026									
Nombre	Apellido	DNI	Email	Telefono					
Eugen	Moga	123456789M	emoga@educacion.navarra.es	600123456					
Paco	Lama	123654987C	paco@prueba.es	612111222					
Maria	Antonia	987654321L	maria@prueba.com	600456789					
Total empleados: 3									

Expresión para agregar la fecha en el informe
="Informe de Personal – " & Today()

Expresion para contar las filas de todos los empleados
="Total empleados: " & CountRows("DataSet1")

Captura de la interfaz informe

InformeEmpl...dlc [Diseño] x MainWindow.xaml MainWindow.xaml.cs

«Expr»



Nombre	Apellido	DNI	Email	Telefono
[Nombre]	[Apellido]	[DNI]	[Email]	[Telefono]
«Expr»				

Para agregar un elemento al pie de página: agregue un elemento al informe y arrástrelo aquí.

Eugen Moga

Captura interfaz de aplicación

Introducción de Datos de Empleados

Introducción de Datos de Empleados

Nombre*

Apellidos*

E-mail*

Teléfono*

Fecha de nacimiento

Seleccione una fecha

DNI*

Adjuntar fotografía

Cargar Foto

Domicilio

Dirección

Ciudad

Provincia

Código Posta

País

Enlace a red social

Rol actual

Descripción del puesto de trabajo actual:

Guardar

Cancelar

Generar Informe

Nombre	Apellidos	DNI	Email	Teléfono
Eugen	Moga	123456789M	emoga@educacion.navarra.es	600123456
Paco	Lama	123654987C	paco@prueba.es	612111222
Maria	Antonia	987654321L	maria@prueba.com	600456789



Captura botón GenerarInforme

```
<Button x:Name="btnGenerarInforme"
        Content="Generar Informe"
        Width="150"
        Height="40"
        Click="btnGenerarInforme_Click"/>
</StackPanel>
```

Eugen Moga

```
// Boton para generar Informe
1 referencia
private void btnGenerarInforme_Click(object sender, RoutedEventArgs e)
{
    reportViewer.LocalReport.ReportPath = "InformeEmpleados.rdlc";

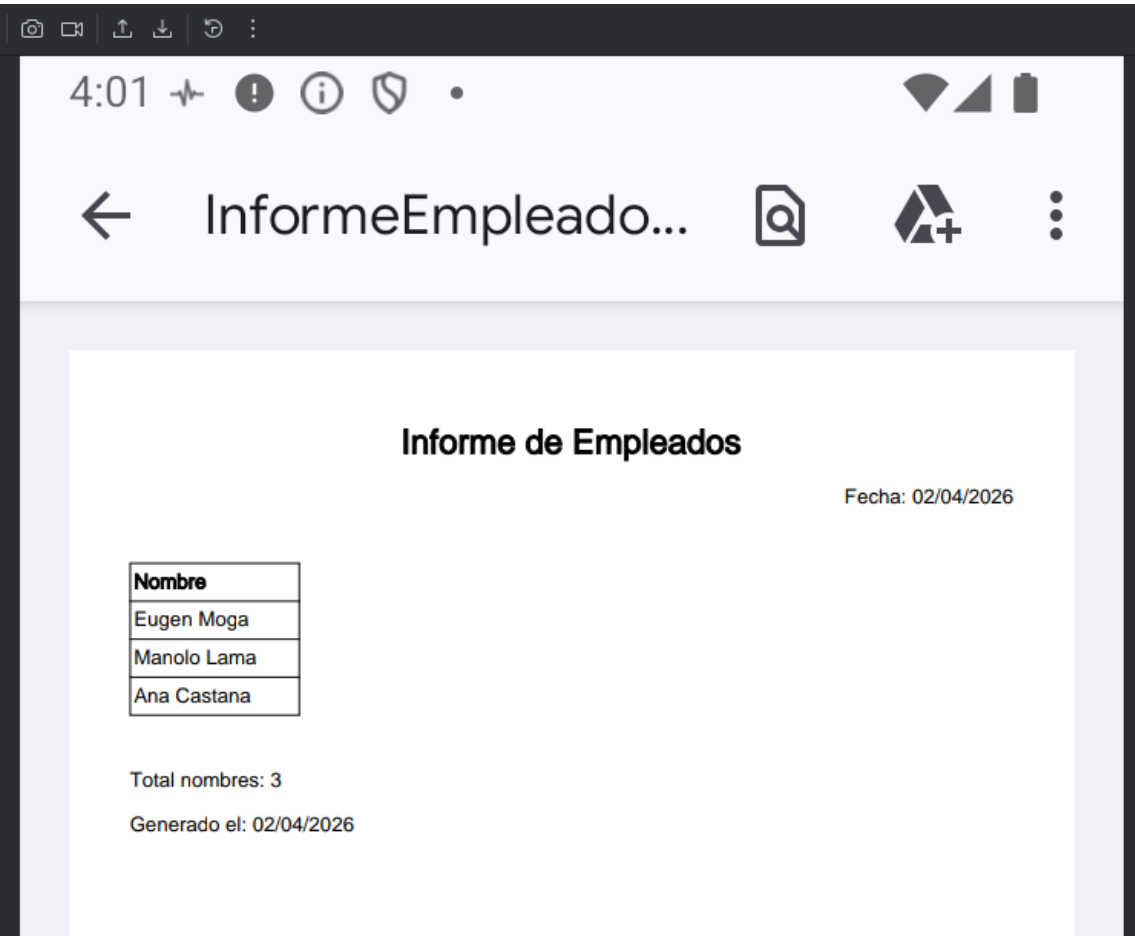
    ReportDataSource rds = new ReportDataSource("DataSet1", listaEmpleados);

    reportViewer.LocalReport.DataSources.Clear();
    reportViewer.LocalReport.DataSources.Add(rds);

    reportViewer.RefreshReport();
}
```

Parte B – Android Informe PDF

Captura del informe PDF abierto en el móvil.



Código de la función generarInformePDF

Como no entra en una sola captura lo agrego por partes explicando un poco el funcionamiento.

Creo la variable dir y con getExternalFile obtengo la ruta de directorio Documentos. Esto lo hago porque me generaba un error de permisos al generar el pdf directamente.

Con File creo el objeto que representa al archivo PDF InformeEmpleados dentro del directorio obtenido anteriormente.

Con PdfWriter creo un escritor de PDF que iText7 usara para escribir datos en el archivo.

Con PdfDocument creo un documento PDF que utiliza el escritor para guardar los datos.

Con Document creo un documento de alto nivel donde puedo añadir los elementos fácilmente (párrafos, tablas, imágenes).

Con SimpleDateFormat obtengo la fecha actual para luego mostrarla en el documento.

```
1 Usage
private fun generarInformePDF(){
    try {
        val dir = getExternalFilesDir( type = android.os.Environment.DIRECTORY_DOCUMENTS)
        val archivo = File( parent = dir, child = "InformeEmpleados.pdf")
        val pdfWriter = PdfWriter( file = archivo)
        val pdfDocument = PdfDocument(pdfWriter)
        val document = Document( pdfDoc = pdfDocument)

        val fechaHoy = SimpleDateFormat( pattern = "dd/MM/yyyy", Locale.getDefault()).format(Date())
    }
}
```

Para la cabecera del documento agrego el titulo centrado y la fecha en la parte derecha del documento.

```
// CABECERA
// Titulo
document.add(Paragraph( text = "Informe de Empleados")
    .setFontSize(20f)
    .setBold()
    .setTextAlignment(TextAlignment.CENTER))

// Fecha
document.add(Paragraph( text = "Fecha: $fechaHoy")
    .setFontSize(12f)
    .setTextAlignment(TextAlignment.RIGHT))
document.add(Paragraph( text = "\n"))
```

Para agregar los datos he creado una tabla y en la cabecera le he puesto Nombre. Después en cada fila he agregado los nombres introducidos en la aplicación.

Aquí tengo que mencionar que he creado un companion object la variable listaNombres para ir guardando los nombres que se introducen en la misma ejecución de la aplicación. Luego agrego todos nombres en la tabla.

Antes en el proyecto solo recibía el nombre a través del intent de MainActivity y lo mostraba en el saludo. Esto al generar el informe solo me mostraba el ultimo nombre introducido en la aplicación.

Por he modificado el proyecto y le he agregado la lista de nombres que guarda los nombres mientras se ejecuta la aplicación. Esto seria interesante hacerlo con una base de datos, pero el enunciado no lo especifica.

```
// DATOS
// Cabecera de la tabla
val tabla = com.itextpdf.layout.element.Table( pointColumnWidths = floatArrayOf(100f))
tabla.addCell(
    headerCell = Cell().add(Paragraph( text = "Nombre").setBold()))

// Fila con nombre recibido
listaNombres.forEach { n ->
    tabla.addCell(Cell().add(Paragraph( text = n)))
}
document.add(tabla)
document.add(Paragraph( text = "\n"))
```

Después de la tabla agrego el pie de pagina donde muestro el numero total de nombres y la fecha cuando se genero el informe. También cierro el documento para evitar posibles errores

```
// PIE DE PAGINA
document.add(Paragraph( text = "Total nombres: ${listaNombres.size}" ))
document.add(Paragraph( text = "Generado el: $fechaHoy" ))

document.close()
```

Eugen Moga

Por último, con FileProvider creo un URI seguro de tipo content:// con permisos temporales para el visor de PDF pueda leer el archivo que se ha creado.

Con Intent le paso el URI para abrir el documento.

```
// Abrir el PDF
val uri = FileProvider.getUriForFile(
    context = this, authority = "${packageName}.fileprovider", file = archivo)

val intent = Intent(action = Intent.ACTION_VIEW).apply {
    setDataAndType(data = uri, type = "application/pdf")
    addFlags(flags = Intent.FLAG_GRANT_READ_URI_PERMISSION)
}
startActivity(intent)
```

Eugen Moga

Todo el código del método generarInformePDF va dentro de un bloque Try – Catch para tratar posibles excepciones.

```
}catch (e: Exception){
    Toast.makeText(context = this, text = "Error: ${e.message}", duration = Toast.LENGTH_LONG).show()
    e.printStackTrace()
}
```

Captura build.gradle.kts



```
</> activity_main.xml </> activity_saludo.xml build.gradle.kts (:app) ×
40 dependencies {
41     implementation(libs.androidx.core.ktx)
42     implementation(libs.androidx.appcompat)
43     implementation(libs.material)
44     implementation(libs.androidx.activity)
45     implementation(libs.androidx.constraintlayout)
46     implementation(libs.androidx.ui.text)
47     testImplementation(libs.junit)
48     androidTestImplementation(libs.androidx.junit)
49     androidTestImplementation(libs.androidx.espresso.core)
50     implementation("com.itextpdf:itext7-core:8.0.5")
51 }
52
```

Eugen Moga

```

Codigo completo del método GenerarInforme
private fun generarInformePDF(){
    try {
        val dir =
getExternalFilesDir(android.os.Environment.DIRECTORY_DOCUMENTS)
        val archivo = File(dir,"InformeEmpleados.pdf")
        val pdfWriter = PdfWriter(archivo)
        val pdfDocument = PdfDocument(pdfWriter)
        val document = Document(pdfDocument)

        val fechaHoy = SimpleDateFormat("dd/MM/yyyy",
Locale.getDefault()).format(Date())

        // CABECERA
        // Titulo
        document.add(Paragraph("Informe de Empleados")
            .setFontSize(20f)
            .setBold()
            .setTextAlignment(TextAlignment.CENTER))

        // Fecha
        document.add(Paragraph("Fecha: $fechaHoy")
            .setFontSize(12f)
            .setTextAlignment(TextAlignment.RIGHT))
        document.add(Paragraph("\n"))

        // DATOS
        // Cabecera de la tabla
        val tabla = com.itextpdf.layout.element.Table(floatArrayOf(100f))
        tabla.addCell(
            Cell().add(Paragraph("Nombre").setBold()))

        // Fila con nombre recibido
        listaNombres.forEach { n ->
            tabla.addCell(Cell().add(Paragraph(n)))
        }
        document.add(tabla)
        document.add(Paragraph("\n"))

        // PIE DE PAGINA
        document.add(Paragraph("Total nombres: ${listaNombres.size}" ))
        document.add(Paragraph("Generado el: $fechaHoy" ))

        document.close()

        // Abrir el PDF
        val uri = FileProvider.getUriForFile(

```

```

        this, "${packageName}.fileprovider", archivo)

    val intent = Intent(Intent.ACTION_VIEW).apply {
        setDataAndType(uri, "application/pdf")
        addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
    }
    startActivity(intent)
} catch (e: Exception){
    Toast.makeText(this, "Error: ${e.message}",
Toast.LENGTH_LONG).show()
    e.printStackTrace()
}
}

```